Python Topic 16 — Decorators

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • Foundation for FastAPI routes, auth guards

🌱 Layer 1 — New to Python

A **decorator** is a function that wraps another function to add extra behavior — without changing the original function's code.

You've already seen them: `@property`, `@staticmethod`, `@classmethod` (Topic 14). The `@` symbol is a decorator.

```
@my_decorator
def say_hello():
    print("Hello!")
```

Like gift-wrapping: the gift (your function) stays the same, but you add wrapping (extra behavior) around it. Common uses: logging, timing, checking permissions, caching.

👩‍💻 Layer 2 — Intermediate (real code + how it works)

Recall: functions are first-class (Topic 10)

```
def greet():
    return "Hi"

f = greet        # assign a function to a variable (no parentheses!)
f()              # "Hi"  → call it through the variable
```

A decorator takes a function and returns a new function

```
def my_decorator(func):
    def wrapper():
        print("Before the function")
        func()                      # call the original function
        print("After the function")
    return wrapper                  # return the wrapped version

def say_hello():
    print("Hello!")
```

```
say_hello = my_decorator(say_hello)  # wrap it
say_hello()
# Before the function / Hello! / After the function
```

The `@` syntax is just shorthand

```
@my_decorator                # this line = say_hello = my_decorator(say_hello)
def say_hello():
    print("Hello!")
```

ℹ️ **The whole magic:** `@my_decorator` is syntactic sugar for `func = decorator(func)`. Nothing more.

Handling arguments with `*args` / `**kwargs` (Topic 10)

```
def my_decorator(func):
    def wrapper(*args, **kwargs):       # accept ANY arguments
        print("Before")
        result = func(*args, **kwargs)  # pass them through
        print("After")
        return result                   # don't forget to return!
    return wrapper

@my_decorator
def add(a, b):
    return a + b

add(3, 5)      # Before / After printed, returns 8
```

🔑 What Are `args` and `kwargs` Actually Holding?

When you call `add(3, 5)`, you're really calling `wrapper(3, 5)` (since `add` is now the wrapped version). Inside `wrapper`:

```
args   = (3, 5)      # a TUPLE of positional arguments
kwargs = {}          # an EMPTY dict — no keyword arguments passed
```

- `args` catches all **positional** arguments → a tuple
- `kwargs` catches all **keyword** arguments → a dict

Then `func(*args, **kwargs)` unpacks them back: `func(*(3, 5), **{})` becomes `add(3, 5)` → returns `8`.

The values change based on how you call it

```
add(3, 5)         # args=(3, 5)      kwargs={}               → add(3, 5)     → 8
add(3, b=5)       # args=(3,)        kwargs={"b": 5}         → add(3, b=5)   → 8
add(a=3, b=5)     # args=()          kwargs={"a":3,"b":5}    → add(a=3, b=5) → 8
```

All three return **8**. The *split* between `args` and `kwargs` differs, but the *reassembled* call is equivalent every time.

The mixed call — one positional, one keyword

For `add(3, b=5)`, the call splits into both:

```
args   = (3,)          # 3 was positional → one-item tuple (note the comma, Topic 6)
kwargs = {"b": 5}      # b=5 was by name → dict
```

Then the wrapper reassembles it:

```
func(*args, **kwargs)
# = func(*(3,), **{"b": 5})
# = add(3, b=5)          ← unpacked back to the original call
# = a=3, b=5 → 3 + 5 → 8
```

```
add(3, b=5)
     │   └──── by name  ──►  kwargs = {"b": 5}
     └──────── by position ►  args   = (3,)

func(*args, **kwargs)  ──►  add(3, b=5)  ──►  8
     (splits apart)          (reassembled)
```

⚠️ **Order rule:** positional arguments must come before keyword arguments. `add(3, b=5)` ✅ is valid; `add(a=3, 5)` ❌ is a SyntaxError ("positional argument follows keyword argument"). `*args, **kwargs` naturally respects this — positionals expand first, then keywords.

✅ **Takeaway:** `*args` / `**kwargs` **split** any call (positional → tuple, keyword → dict); `func(*args, **kwargs)` **glues it back** to the original call. This is what makes a decorator work on any function, called any way.

🧠 Layer 3 — Expert (internals & gotchas)

1. Always use `functools.wraps`

```
from functools import wraps

def my_decorator(func):
    @wraps(func)                    # ← preserves func's name, docstring, etc.
    def wrapper(*args, **kwargs):
```

```
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def greet():
    """Say hello."""
    pass

greet.__name__      # "greet"    (without @wraps → "wrapper" 😬)
greet.__doc__       # "Say hello." (without @wraps → None)
```

🚨 **Note:** without `@wraps`, the decorated function loses its real name and docstring (they become `wrapper`'s). Frameworks and debuggers rely on it — always add `@wraps(func)`.

2. A real, useful decorator — timing

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)

slow_function()      # slow_function took 1.0012s
```

3. Decorators WITH arguments — one extra layer of nesting

```
def repeat(times):                      # takes the decorator's argument
    def decorator(func):                # takes the function
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
```

```
    return decorator

@repeat(times=3)                        # note the parentheses — it's called!
def greet():
    print("Hi")

greet()      # prints "Hi" three times
```

ℹ️ **Rule:** a decorator *with* arguments needs **three** nested functions — outer takes the arguments, middle takes the function, inner is the wrapper. Exactly how `@app.get("/path")` works in FastAPI.

4. Stacking decorators — applied bottom-up

```
@timer
@my_decorator
def process():
    ...

# equivalent to: process = timer(my_decorator(process))
# applied bottom-to-top: my_decorator first, then timer wraps that
```

5. Common built-in / library decorators

```
from functools import lru_cache

@lru_cache(maxsize=None)         # caches results — huge for expensive/recursive calls
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# You'll see these constantly:
@property          # Topic 14 — computed attribute
@app.get("/users") # FastAPI route (decorator with args)
@pytest.fixture    # pytest test setup
@dataclass         # auto-generates __init__, __repr__, __eq__
```

✅ **Free win:** `@lru_cache` memoizes results so repeated calls with the same arguments are instant. Great for expensive computations and recursion.

6. `@dataclass` — auto-generates boilerplate

```
from dataclasses import dataclass
```

```python
@dataclass
class Point:
    x: int
    y: int

# automatically gives __init__, __repr__, __eq__ for free!
p = Point(3, 5)
print(p)            # Point(x=3, y=5)   → free __repr__
Point(3, 5) == Point(3, 5)   # True → free __eq__
```

Writes `__init__`, `__repr__`, `__eq__` for you from the type-annotated fields. The modern way to make data-holding classes with far less boilerplate. Pydantic (FastAPI) is a supercharged version of this idea.

🎯 Interview Q&A Cards

Q: What is a decorator?

A: A function that takes a function and returns a wrapped version, adding behavior without modifying the original. `@decorator` is sugar for `func = decorator(func)`.

Q: In `wrapper(*args, **kwargs)`, what do they hold for `add(3, b=5)`?

A: `args = (3,)` (positional → tuple), `kwargs = {"b": 5}` (keyword → dict). `func(*args, **kwargs)` reassembles the call as `add(3, b=5)` → 8.

Q: Why use `functools.wraps`?

A: It copies the original function's name, docstring, and metadata onto the wrapper. Without it, introspection shows `wrapper`, breaking debuggers, docs, and some frameworks.

Q: How does a decorator that takes arguments work?

A: Three nested functions — outer receives the arguments and returns the decorator, which receives the function and returns the wrapper. That's how `@app.get("/path")` is built.

🪝 **Memory hook:** *`@decorator` = `func = decorator(func)`. It wraps a function to add behavior. `*args` / `**kwargs` split a call, `func(*args, **kwargs)` glues it back. Always `@wraps(func)`. With arguments = three nested functions.*

✅ **One-liner for interviews:** "A decorator is a function that takes a function and returns a wrapped version, adding behavior without modifying the original. `@decorator` is sugar for `func = decorator(func)`. Use `*args/**kwargs` to wrap any signature and `functools.wraps` to preserve metadata. Decorators with arguments need three nested layers — that's how FastAPI routes work."